

A Failed Diagnosis of Entropy Collapse in Small-Budget MAPPO for Multi-Robot Area Coverage

Harsh Pandhe¹

¹Independent Research , harshpandhehome@gmail.com

October 2026

Abstract

In our serial Gazebo setup, a single multi-robot training experiment is limited to thousands rather than millions of environment steps. In an early run of a centralized-critic MAPPO policy for three-robot area coverage (15 iterations, 4,500 steps, RLlib default GAE $\lambda = 1.0$ and entropy coefficient 0.0), we recorded policy entropy falling to -0.53 and episode returns between -227 and $+3$, and attributed this to the default hyperparameters. This paper is a reproducibility companion to a separate negative-result study [6]. We proposed a correction (smaller λ , a nonzero entropy bonus, a larger batch, shorter episodes and a rebalanced reward). The raw logs of that run were not kept, and the attribution was never tested against a control. Here we test it: six training runs treated as independent replicates, three per configuration, with per-iteration metrics logged. We did not observe the historical entropy-collapse pattern. Under a reconstruction of the original configuration, entropy at iteration 15 was 1.75, 1.50 and 1.87 (minimum 0.79 in one run, which then recovered), nowhere near -0.53 . Under the corrected configuration it was 1.95, 2.12 and 2.48, higher in all three runs, although it also received $2.4\times$ as many environment steps (entropy rose by 0.36 to 0.90 over training, against -0.45 to $+0.48$ for the reconstruction), but with three runs per arm the smallest attainable two-sided permutation p is 0.10, which is also the value obtained. Pooled per-episode return SDs were 72 and 67 (descriptive only), and mean returns were negative throughout and did not clearly improve in either arm. The comparison changes five things at once and logs no task metric, so it cannot say which change matters or whether the correction improves coverage. We conclude that the original diagnosis, built from one unreplicated run, is not supported, that run-to-run

variability in entropy is large enough for a single run to mislead, and that the corrected configuration raises entropy modestly without our having shown a benefit.

Keywords: multi-agent reinforcement learning, MAPPO, entropy collapse, environment-step budget, reproducibility, negative results

1 Introduction

Work on sample efficiency in deep reinforcement learning usually assumes that environment steps are cheap or parallelizable. General-purpose robotics simulators used for sensor-level fidelity (LiDAR ray casting, collision geometry, ROS integration) are often run as a single real-time instance, which, in our setup, makes a budget of thousands of environment steps per experiment a practical ceiling rather than a choice. Under such budgets a batch contains only a few complete episodes, and the usual intuitions about advantage-estimation variance and exploration pressure are worth examining.

This paper began as a diagnosis. After one early training run showed what looked like entropy collapse, we traced it, by reasoning about the configuration, to RLlib’s default $\lambda = 1.0$ and entropy coefficient 0.0, proposed a correction, and wrote it up as a mechanistic case study. It is now a report on testing that diagnosis. We ran the control we should have run first, and the result does not support the original account. We report it because the failure to reproduce is informative: it shows how a causal story assembled from a single unreplicated run can look convincing and be wrong, and it gives a three-run baseline for entropy trajectories under the reconstructed configuration. The claim is narrow. We do not claim that default hyperparameters are safe in small-batch multi-agent training in general, only that in this system, with three runs per arm, we did not see the collapse we reported.

2 Related Work

Yu et al. [9] establish MAPPO with a centralized critic as a strong baseline for cooperative multi-agent games; their results are reported with far more environment steps than are available here. Work aimed at closing the sample-budget gap includes differentiable surrogate models alongside a non-differentiable simulator [10] and world-model-based learning without RL [8], which require a learned or differentiable dynamics model that we do not assume. Masked reconstruction as an auxiliary task improves sample efficiency in value-based MARL [5], and physics-informed structure has been used for distributed multi-robot learning [7]; both are complementary to the hyperparameter-level question studied here.

Henderson et al. [4] show that results in deep RL depend heavily on random seeds, implementation details and the number of runs, which is the concern illustrated by our repeated-run experiment. Colas et al. [3] and Agarwal et al. [1] argue for power analysis and interval estimates when few runs are available, and Andrychowicz et al. [2] show that on-policy implementation choices (including advantage estimation and entropy regularization) can change outcomes substantially; our small test is in that spirit but far smaller in scale. A companion study [6] evaluates the corrected configuration’s downstream coverage performance against a classical planner.

3 System and the Original Configuration

The system is a shared MAPPO policy for three TurtleBot3 robots performing cooperative area coverage in Gazebo Harmonic (ROS 2 Jazzy), trained with Ray RLlib. Each robot observes a 46-dimensional vector: 24 LiDAR ranges, goal distance and bearing, linear and angular velocity, and nine neighbor slots of (distance, bearing). Each neighbor pair is embedded by a two-layer ReLU network ($2 \rightarrow 16 \rightarrow 16$), masked and mean-pooled, concatenated with the 28 ego features, and passed through a two-layer Tanh actor ($44 \rightarrow 64 \rightarrow 64$). A centralized critic ($76 \rightarrow 64 \rightarrow 64 \rightarrow 1$, Tanh) takes the same 44 features plus a mean-pooled embedding ($48 \rightarrow 32 \rightarrow 32$, ReLU) of the other agents’ observation–action pairs. Training runs with `num_env_runners=0`, since Gazebo must run in the main process, so all rollouts are serial. Learning rate 10^{-4} , PPO clip 0.2, $\gamma = 0.99$, 5 epochs and minibatch size 64 are common to all runs. Runs used code commit `1a83e72`, an AMD Ryzen 5 7600 (6 cores, 12 threads), about 30 GB RAM and Ubuntu 24.04.5 LTS. Wall-clock time was about 1,010 s per original-arm run and 2,508 s per corrected-arm run.

Entropy metric and “collapse. The policy has two continuous action dimensions (v, ω) . Training uses Ray RLlib 2.55.1 on the old (ModelV2) API stack with the PyTorch framework. For a box action space RLlib selects `TorchDiagGaussian`, whose entropy (per its source) is the per-dimension Gaussian entropy summed over the two action dimensions; PPO logs its mean over the training batch (all agents and timesteps in the batch, with the shared policy). It is the *differential* entropy of the unsquashed Gaussian, before action clipping or normalization, so it can be negative and a negative value is not in itself pathological. We therefore use “entropy collapse” operationally to mean a sharp fall of logged entropy relative to the run’s earlier values, not the sign of the value. Because the historical run lacks raw logs, we cannot retrospectively define a quantitative threshold; “collapse” is a descriptive label for the historical trajectory, not a formally thresholded outcome; the historical -0.53 is simply an unusually low observed value, with no demonstrated pathological threshold.

Original observation. The first configuration left GAE λ and the entropy coefficient at RLib’s defaults (1.0 and 0.0), used a train batch of 300 steps, 150-step episodes, a collision penalty of -20 and a coverage weight of 2.0. In one 15-iteration run (4,500 steps) we recorded final-iteration entropy of -0.53 and episode returns between -227 and $+3$. These figures come from our notes of that run; its logs were not archived, and we cannot re-derive them. Because the original code state is not archived, the arm called “original below is a *reconstruction* of the described settings, not the original run. The original account attributed the entropy to two mechanisms: a $\lambda = 1.0$ Monte-Carlo estimator is high-variance when a batch holds only a few episodes, and a zero entropy coefficient exerts no pressure against premature convergence. Neither mechanism was tested. We also stated that a batch held “three to four” episodes; in the runs reported below it held two.

Proposed correction. $\lambda = 0.95$; entropy coefficient 0.01; train batch 700 with 90-step episodes (more distinct episodes per iteration at the same nominal iteration count; total steps and wall-clock time are reported below); collision penalty -8 and coverage weight 4.0. We considered but did not pursue curriculum learning and behavior-cloning warm starts.

4 Controlled Test

Questions and scope. (1) Does the reconstructed original configuration reproduce the collapse? Mostly no. (2) Does the five-change correction raise entropy? Numerically yes, not established statistically. (3) Which change caused any difference? Unknown. (4) Does higher entropy give better coverage? Not tested here; the companion study [6] addresses downstream task performance, and this paper is restricted to the training-dynamics diagnosis.

Design. Two configurations, *reconstructed original* and *corrected*, each trained for 15 iterations, three runs treated as independent per configuration, interleaved in time (original, corrected, original, ...) so that any drift in the simulator or host affects both; interleaving reduces, but does not eliminate, possible temporal or host dependence between runs. Everything not listed in Section 3 is identical, including code and world. The Gazebo seed was fixed and RLib’s seed was not set, so runs differ through RLib’s own randomness and simulator timing; they are not intended to be bitwise-reproducible, and the aim is to characterize variability under repeated launches. The experiment was resource-constrained and was intended to establish whether the historical behavior was readily reproducible, not to give a high-powered estimate of a treatment effect. Per-iteration policy entropy, losses, value-function explained variance and episode returns were logged. The unit of independence is the training run, so $n = 3$ per arm. The analysis plan was fixed before we examined the

outcomes of the six controlled runs: the primary outcome is entropy at iteration 15 (original lower than corrected); the secondary outcome is the change in entropy from iteration 1 to 15; return spread is descriptive only because return scales differ between arms. One change was made after the first run finished: we replaced the within-iteration range of returns, which grows with the number of episodes (two against eight per iteration), with the standard deviation of individual episode returns. Tests are exact two-sided permutation tests on the difference of arm means. With three runs per arm there are 20 permutations, so the smallest attainable p is 0.10.

What the comparison cannot separate. The two configurations differ in λ , entropy coefficient, batch size, episode length and reward scale, and they collect different numbers of steps (4,500 against 10,800, because sampling ends on whole episodes; the nominal 15×700 is 10,500). We therefore cannot attribute any difference to a single change; the comparison tests a configuration-level intervention, not any individual hyperparameter mechanism. No coverage or other task metric was logged and the checkpoints were not retained, so we say nothing here about task performance.

Table 1: Six 15-iteration training runs. Entropy is the RLlib-reported batch-mean differential entropy, summed over the two continuous action dimensions (Section 3); Δ is iteration 15 minus iteration 1. Return columns are over the new episodes of each iteration (30 episodes per original run, about 120 per corrected run). Return scales differ between configurations.

Configuration	Run	Ent. it. 1	Ent. it. 15	Δ	Ent. min	Mean return	Return SD	Steps
Reconstructed original	1	1.61	1.75	+0.14	1.43	-59	66	4,500
Reconstructed original	2	1.95	1.50	-0.45	0.79	-62	67	4,500
Reconstructed original	3	1.39	1.87	+0.48	1.39	-66	84	4,500
Corrected config	1	1.59	1.95	+0.36	1.43	-43	61	10,800
Corrected config	2	1.59	2.12	+0.53	1.50	-51	69	10,800
Corrected config	3	1.58	2.48	+0.90	1.58	-56	71	10,800

4.1 Results

The historical entropy-collapse pattern did not recur in the reconstructed configuration. Entropy at iteration 15 was 1.75, 1.50 and 1.87 (mean 1.71). The lowest value in any original run was 0.79 (run 2, iteration 12), after which it rose to 1.50. No run approached the reported -0.53.

The corrected configuration gave numerically higher entropy, not statistically distinguishable at $n = 3$. Entropy at iteration 15 was 1.95, 2.12 and 2.48 (mean 2.18); the difference in means is -0.475 (original minus corrected), exact $p = 0.10$, the smallest value attainable at $n = 3$. The difference in means is +0.48 in favour of the corrected

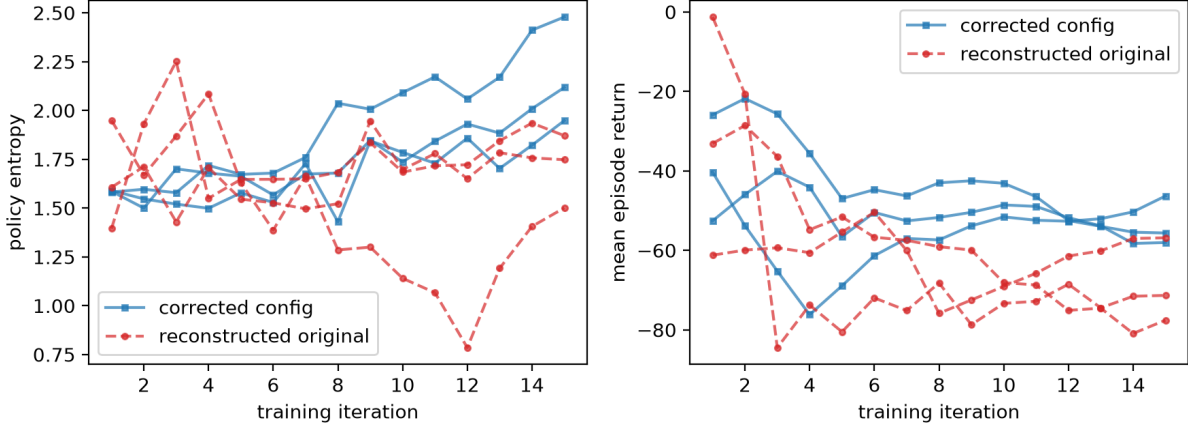


Figure 1: Policy entropy (left) and mean episode return per iteration (right) for the six runs, three per configuration. The reconstructed original (red, dashed) does not collapse; corrected runs are solid blue; one run dips to 0.79 at iteration 12 and recovers.

arm, with a descriptive, model-based 95% Welch confidence interval of $[-0.08, 1.03]$ (Welch–Satterthwaite $df \approx 3.6$); the interval is wide and includes zero, and it rests on normality assumptions that three runs cannot check. Because the corrected arm also received $2.4\times$ as many environment steps by iteration 15, any entropy difference cannot be separated from additional training exposure, let alone attributed to a single hyperparameter. Entropy rose over training in all three corrected runs ($+0.36, +0.53, +0.90$) and was mixed in the original runs ($+0.14, -0.45, +0.48$); the difference in the change is -0.54 , $p = 0.20$. Mean entropy over the 15 iterations differed by only -0.16 ($p = 0.20$). Because the corrected configuration explicitly adds an entropy bonus, its higher logged entropy is partly an expected consequence of the modified objective rather than evidence of improved exploration or learning.

Return spread (descriptive). The standard deviation of per-episode returns was 72 (reconstructed original, pooled) against 67 (corrected); run-level values were 66, 67, 84 and 61, 69, 71. This is descriptive only and we do not treat it as confirmatory. Individual episode returns reached about -350 in both arms (the historical note recorded -227 to $+3$, a different range from the one observed now), so large return swings also occurred in the corrected configuration and cannot be regarded as unique to the reconstructed defaults in these runs. Mean returns did not improve with training. Comparing the last five iterations with the first five, mean return was lower in four of the six runs, essentially unchanged in one (original run 2, $-59 \rightarrow -60$) and higher in one (corrected run 3, $-61 \rightarrow -51$). Averaged over all iterations the original runs gave $-59, -62$ and -66 and the corrected runs $-43, -51$ and -56 , but the arms use different collision penalties and episode lengths, so we do not compare these directly.

5 Discussion

The entropy-collapse diagnosis was not reproduced under a reconstructed version of the recorded original configuration. The mechanisms it proposed remain plausible in principle, but the symptom they were meant to explain, entropy collapse under the defaults, did not appear in three attempts. Two explanations are consistent with this. The original run may have been an unrepresentative draw: the original arm’s entropy trajectories are variable (final values 1.50 to 1.87, one transient dip to 0.79), so a single run can look like collapse or like nothing. Or the early run differed from these in some respect we did not record, since the code changed between them in ways not captured in the notes. We cannot tell these apart, and we do not claim the first.

The corrected configuration raises entropy modestly and, in this sample, consistently in direction, but the sample is too small to call it an effect, and we did not measure whether it improves the task. The companion study [6] trained the corrected configuration for 45 iterations and found it did not close the gap to a classical planner, and that two reward-coefficient variants of it did not produce a statistically significant change in coverage at three seeds. Taken together, the entropy correction is at most a stabilizing measure, not a demonstrated route to a competent policy.

The practical point is about method. Entropy is cheap to log per iteration, and three replicate runs per configuration cost a few hours here. Had we done that before writing the diagnosis, we would not have published a causal account of a pattern that does not replicate.

6 What a Proper Follow-Up Would Require

Change one factor at a time (e.g. λ alone, entropy coefficient alone) at an equal environment-step budget; log a task metric such as area-coverage rate each iteration; set RLib seeds explicitly; use at least five runs per arm; and retain checkpoints so that policies can be evaluated afterwards.

7 Limitations

Three runs per arm is small: the smallest attainable p is 0.10, so no comparison here can reach conventional significance, and we report the values instead. The comparison bundles five changes and cannot attribute an effect to any. Fifteen iterations is short, and entropy might diverge later; the original claim concerned iteration 15. We tested one environment and one architecture. The simulator’s timing is not deterministic, so the runs differ from one

another for reasons we do not control, which is part of what a replicate measures. RLlib’s seed was unset, and the two arms differ by $2.4\times$ in steps. We did not log a task metric or keep the checkpoints, so we make no claim about coverage. The original run’s logs are unavailable, so the failure to reproduce it cannot be traced to a cause.

8 Conclusion

A diagnosis of entropy collapse from default RLlib hyperparameters, formed from one unreplicated run, was not observed in three runs of a reconstructed version of the recorded configuration: entropy at iteration 15 stayed between 1.50 and 1.87. The corrected configuration raised entropy modestly in all three runs without reaching significance at the attainable sample size, and we did not measure a task benefit. We report the failure to reproduce, the six-run entropy measurements and trajectories and the scripts that produced them, in the hope that the baseline is more useful than the original diagnosis was.

Data and code availability. Per-iteration logs for all six runs, the analysis summary and the scripts are in the project repository (analysis artifacts under `checkpoints/paper2_ab/`; trained model checkpoints were not retained) and `scripts/` (files `paper2_ab_entropy.sh` and `paper2_ab_analyze.py`), archived at <https://doi.org/10.5281/zenodo.22758286>.

A Infrastructure Faults

Two distinct faults occurred during this project and are easy to conflate. First, a native segmentation fault in `torch.optim.Adam.__init__`; an earlier draft attributed it to a local installation fault, but it was a TorchDynamo compilation hook failing inside Ray’s actor threads and was resolved by disabling TorchDynamo (`TORCHDYNAMO_DISABLE=1` together with the in-process setting). Second, a later training run stopped at iteration 34 of 45 when a host Wi-Fi interruption severed Ray’s control plane, which had bound to the host’s dynamic address; binding Ray to the loopback interface and capping its object store in-process resolved it. Neither affects the six runs here.

References

- [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

- [2] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters in on-policy reinforcement learning? a large-scale empirical study, 2020.
- [3] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? statistical power analysis in deep reinforcement learning experiments, 2018.
- [4] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3207–3214, 2018.
- [5] Jung In Kim, Young Jae Lee, Jongkook Heo, Jinhyeok Park, Jaehoon Kim, Sae Rin Lim, Jinyong Jeong, and Seoung Bum Kim. Sample-efficient multi-agent reinforcement learning with masked reconstruction. *PLoS ONE*, 18(9):e0291545, 2023.
- [6] Harsh Pandhe. Diagnosing a low-displacement failure mode in MAPPO for multi-robot area coverage. SSRN preprint, abstract id 7556699 (under review); code and data: <https://doi.org/10.5281/zenodo.22758286>, 2026.
- [7] Eduardo Sebastian, Thai Duong, Nikolay Atanasov, Eduardo Montijano, and Carlos Sagues. Physics-informed multi-agent reinforcement learning for distributed multi-robot problems. *IEEE Transactions on Robotics*, 41, 2025.
- [8] Haojie Shi, Tingguang Li, Qingxu Zhu, Jiapeng Sheng, Lei Han, and Max Q.-H. Meng. An efficient model-based approach on learning agile motor skills without reinforcement, 2024. Accepted to ICRA 2024.
- [9] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- [10] Yuang Zhang, Yunlong Song, Zhihao He, Zelin Ni, Kangyu Wang, Tianchi Liu, Yu Hu, Feng Yu, Danping Zou, and Weiyao Lin. Asymmetric physics enables efficient learning in quadrupedal robot swarms, 2026.